

EdgeFlow 用户手册

边缘计算平台使用说明书

版本：v0.1.0 (MVP)

编制日期：2026 年 8 月 14 日

适用产品：EdgeFlow v0.1.0

本文档随产品发布，内容与 v0.1.0 开发进展一致；

未实现功能已标注”即将上线”或列入附录 E 已知限制。

版本信息

项目	内容
产品名称	EdgeFlow（边缘计算平台）
手册版本	v0.1.0
产品版本	v0.1.0（2026-08-14 发布）
编制日期	2026 年 8 月 14 日
适用读者	平台安装运维人员、边缘节点操作人员、应用部署人员
编译方式	XeLaTeX（ctex 文档类），见 README.md
变更记录	见附录 D

目录

版本信息	2
第一章 产品概述	6
1.1 EdgeFlow 是什么	6
1.2 核心组件	6
1.3 典型应用场景	7
1.4 版本与发布（v0.1.0）	7
1.5 术语约定	8
1.6 功能边界与已知限制	8
1.7 手册内容导航	8
第二章 环境要求与安装部署	10
2.1 环境要求	10
2.2 获取软件	10
2.2.1 方式一：源码构建	10
2.2.2 方式二：使用发布制品	10
2.2.3 方式三：容器镜像	11
2.3 安装云端（cloudcore）	11
2.3.1 方式一：keadm 生成 Kubernetes 部署产物（推荐）	11
2.3.2 方式二：Helm 部署	12
2.3.3 方式三：裸进程运行（单机联调）	12

2.4	接入边缘节点 (edgecore)	12
2.5	部署验证	14
2.6	卸载与清理	14
2.7	升级与回滚	14
2.8	常见问题	15
第三章	快速入门	16
3.1	准备工作	16
3.2	第一步：构建制品	16
3.3	第二步：启动云端 cloudcore	16
3.4	第三步：启动边缘 edgecore	17
3.5	第四步：验证节点注册	17
3.6	第五步：下发第一个 Pod (nginx)	17
3.7	第六步：查看容器与 Pod 状态	17
3.8	第七步：查看设备数据	18
3.9	第八步：下发设备指令	18
3.10	一键 Demo (可选)	18
3.11	清理	18
3.12	常见问题	19
第四章	云端操作指南	20
4.1	通用约定	20
4.2	健康检查	22
4.2.1	GET /healthz	22
4.3	节点 API	22
4.3.1	GET /api/v1/nodes ——全部节点 (运行视角)	22
4.3.2	GET /api/v1/nodes/{nodeID} ——单节点详情	23
4.3.3	GET /api/v1/edgenodes ——全部节点 (CRD 视角)	23
4.3.4	GET /api/v1/edgenodes/{nodeID} ——单节点 EdgeNode 对象	24
4.4	Pod API	24
4.4.1	GET /api/v1/pods ——全部 Pod 状态	24
4.4.2	GET /api/v1/nodes/{nodeID}/pods ——单节点 Pod 状态	24
4.5	设备 API	25
4.5.1	GET /api/v1/devices ——全部设备状态	25
4.5.2	GET /api/v1/nodes/{nodeID}/devices ——单节点设备状态	25
4.6	下发类 API (可靠下发)	25
4.6.1	POST /api/v1/nodes/{nodeID}/podsync ——下发 Pod 配置	25
4.6.2	POST /api/v1/nodes/{nodeID}/config-sync ——下发配置	26
4.6.3	POST /api/v1/nodes/{nodeID}/device-command ——下发设备指令	26
4.6.4	五态响应语义	27

4.7	监控指标	27
4.7.1	GET /metrics	27
4.8	API 认证	28
4.8.1	启用认证	28
4.8.2	调用方式	28
4.9	排障指南	29
第五章	边缘节点与自治运行	30
5.1	接入与连接管理	30
5.1.1	注册	30
5.1.2	心跳保活	30
5.1.3	断线重连	30
5.2	可靠投递	31
5.3	Edged 应用调谐	31
5.3.1	多副本 Pod	31
5.3.2	健康自愈与 CrashLoopBackOff	31
5.3.3	镜像漂移检测	32
5.4	断网自治	32
5.5	edgecore 环境变量配置	32
第六章	设备接入	34
6.1	设备模型：影子	34
6.2	设备指令下发链路	34
6.3	Mapper 框架	35
6.3.1	mock_sensor：模拟传感器	35
6.3.2	Modbus TCP Mapper	35
6.3.3	Mapper 的两种工作模式	35
6.4	EventBus 数据面与降级	36
6.5	设备状态查询	36
6.6	设备接入检查清单	36
第七章	升级与回滚	38
7.1	机制总览	38
7.2	升级前准备	38
7.3	升级：keadm upgrade	38
7.4	回滚：keadm rollback	39
7.5	台账查询：keadm ops-ledger	40
7.6	Helm 路径（云端组件）	40
7.7	端到端演练建议	40
第八章	安全与认证	42

8.1	云边通道 mTLS	42
8.1.1	启用开关	42
8.1.2	使用 keadm 生成 TLS 产物	42
8.2	证书管理	43
8.2.1	证书布局	43
8.2.2	轮换（人工编排）	43
8.3	管理 API Token 认证	43
8.4	审计台账	44
8.5	安全基线小结	44
第九章	常见问题与故障排查	45
9.1	日志位置	45
9.2	常见问题速查表	45
9.3	分层排查指引	45
9.3.1	云边通道层	45
9.3.2	容器运行层	45
9.3.3	设备数据面层	47
9.4	API 错误语义速记	47
9.5	环境相关的已知边界	47
附录 A	附录 A 环境变量参考	48
A.1	cloudcore 组	48
A.2	edgecore 组	48
A.3	keadm / 开发脚本组	50
附录 B	附录 B API 状态码语义	51
附录 C	附录 C 术语表	53
附录 D	附录 D 版本与变更记录	54
D.1	v0.1.0（2026-08-14，MVP 首发）	54
D.2	已实现 vs 即将上线 / 范围外	54
附录 E	附录 E 已知限制与边界	55

第1章 产品概述

本章内容：介绍 EdgeFlow 边缘计算平台（Edge Computing Platform）的定位、核心组件、典型应用场景、版本发布情况与使用边界，帮助操作人员建立整体认识。

1.1 EdgeFlow 是什么

EdgeFlow 是一个面向“云—边—设备”三层架构的轻量级边缘计算平台，当前版本为 v0.1.0。它通过云端统一接口，让操作人员可以在云端完成边缘节点的接入与管理、容器应用的下发与运行、设备的接入与远程控制，并实时掌握边缘侧的运行状态。

EdgeFlow 的典型工作方式：边缘节点上的 **edgecore** 主动连接云端 **cloudcore** 并保持长连接；操作人员通过云端 REST API（Representational State Transfer Application Programming Interface）下发期望（desired）配置；边缘节点执行后，将实际状态（reported）上报云端，形成“下发—执行—上报”的闭环。整个链路包括节点注册、容器调谐（Reconcile）、设备数字孪生（Digital Twin）与配置同步等能力。

1.2 核心组件

EdgeFlow v0.1.0 由以下组件构成：

组件	角色	职责
cloudcore	云端控制面	对外提供 REST API（默认端口 8080）；通过 CloudHub（WebSocket，默认端口 10000）与边缘节点保持长连接，处理节点注册、心跳、可靠下发与设备指令
edgecore	边缘运行时	部署在边缘节点上，由五个子模块协作完成云边通道、元数据管理、容器调谐、设备孪生与事件总线（详见下文）
keadm	安装管理 CLI	生成云端与边缘的部署产物，支持 <code>init/join/reset/upgrade/rollback/ops-ledger</code> 命令
mock-cloudhub	联调工具	在本地模拟云端 CloudHub，用于单独调试 edgecore 的注册、心跳与重连行为
mappers（Mapper 设备接入程序）	设备接入	将物理或模拟设备接入平台：内置 <code>mock_sensor</code> 模拟温湿度传感器（设备名 <code>sensor-01</code> ）， <code>modbus</code> 支持 Modbus TCP 协议设备（默认设备名 <code>mb-sensor-01</code> ）

其中，**edgecore** 由以下子模块组成：

- **EdgeHub**: 云边通信通道, 与云端 CloudHub 建立 WebSocket 长连接, 负责消息收发;
- **MetaManager**: 元数据管理, 使用 SQLite 将边缘侧元数据持久化落盘;
- **Edged**: 容器调谐, 基于 Docker 运行时保证容器实际状态与期望状态一致;
- **DeviceTwin**: 设备数字孪生, 维护设备期望值 (desired) 与实际上报值 (reported);
- **EventBus**: 事件总线, 基于 MQTT 提供设备遥测与指令的数据面通道。

1.3 典型应用场景

- **边缘容器应用部署**: 在云端下发 nginx 等容器镜像, 由边缘节点本地运行, 业务数据不出边缘;
- **设备数据采集与数字孪生**: 温湿度传感器周期上报属性, 云端可随时查看设备实时数据;
- **设备远程控制**: 通过设备指令下发期望值 (如目标温度), 由 Mapper 在边缘执行;
- **边缘节点管理**: 查看节点注册状态、心跳与平台信息, 感知节点在线/离线状态;
- **配置下发**: 将 ConfigMap/Secret 配置同步到边缘节点, 供边缘应用使用。

1.4 版本与发布 (v0.1.0)

- **版本**: v0.1.0 (2026-08-14 发布);
- **容器镜像**: edgeflow/cloudcore:v0.1.0、edgeflow/edgecore:v0.1.0, 支持 linux/amd64 与 linux/arm64 双架构;
- **Helm Chart**: build/charts/edgeflow, 用于在 Kubernetes 集群中部署云端;
- **离线制品**: 发布目录 `release/v0.1.0/`, 包含云/边/管理工具在各平台的二进制、校验和 (checksums.txt)、软件物料清单 (SBOM) 与 Helm Chart 包 (edgeflow-0.1.0.tgz)。

1.5 术语约定

术语	说明
nodeID	边缘节点唯一标识。edgecore 注册时上报，默认格式为 edge- <code>< 主机名 ></code> ，仅允许字母、数字、连字符与下划线
Pod	云端下发的容器应用单元，包含名称 (name)、命名空间 (namespace)、镜像 (image) 与副本数 (replicas) 等描述
Mapper	设备接入程序，负责协议解析、属性采集与指令执行（如 <code>mock_sensor</code> 、 <code>modbus</code> ）
数字孪生 (Digital Twin)	设备的云端影子: <code>desired</code> 为云端下发的期望值, <code>reported</code> 为设备实际上报值
可靠下发 (Reliable Delivery)	下发类接口的消息投递机制: 消息送达边缘并收到确认 (Ack) 后才返回成功, 未确认则超时重试
调谐 (Reconcile)	边缘侧周期性比对期望状态与实际状态, 并驱动实际状态收敛到期望状态的过程

1.6 功能边界与已知限制

使用 EdgeFlow v0.1.0 前, 请了解以下边界:

- **云端状态为内存态:** cloudcore 保存的节点、Pod、设备状态不落盘, 重启后查询数据清空; 边缘节点重连后会自动重新注册并恢复上报, 边缘侧元数据由 SQLite 持久化, 不受影响;
- **接入令牌为预留字段:** `keadm join` 的 `--token` 参数当前版本尚未被 edgecore 消费, 用于后续版本的接入鉴权;
- **设备指令值类型:** `device-command` 的 `value` 为数值型 (float64), 字符串、布尔型属性暂不支持通过该端点下发;
- **Secret 明文存储:** `config-sync` 下发 Secret 时值为明文, 生产环境需自行加密;
- **无批量下发端点:** 当前版本未提供面向多节点的批量 (广播) 下发接口。

1.7 手册内容导航

本手册面向操作人员, 共分若干章节:

- 第 2 章 [chapter 二](#): 环境要求与安装部署——云端与边缘节点的安装、验证、卸载与升级回滚;
- 第 3 章 [chapter 三](#): 快速入门——单机跑通完整链路;
- 第 4 章 [chapter 四](#): 云端操作指南——REST API、监控指标与认证配置;

- 后续章节：设备接入（Mapper）使用、边缘应用管理、维护与排障等专题。

第2章 环境要求与安装部署

本章内容：说明 EdgeFlow v0.1.0 的运行环境要求，并给出完整的安装部署步骤，包括获取软件、使用 `keadm` 初始化云端与接入边缘节点、Helm 部署，以及部署验证、卸载清理与升级回滚。

2.1 环境要求

部署角色	环境要求
云端 (cloudcore)	支持 linux/amd64 与 linux/arm64；建议 2 核 4 GB 以上；云端状态为内存态，无需持久化存储
Kubernetes 集群	使用 <code>keadm</code> 产物或 Helm 部署时需准备集群；需要 <code>kubect1</code> （与集群版本兼容）与 Helm v3+
边缘节点 (edgecore)	Linux 系统（支持 systemd）；Docker daemon 运行中（Edged 的容器运行时）；可访问云端 CloudHub 端口（默认 10000，NodePort 部署时为集群分配的节点端口）；SQLite 数据目录可写
管理机 (keadm)	任意 amd64/arm64 主机，仅用于生成离线部署产物，不直接操作集群
源码构建	Go 1.26+ 与 Make；构建容器镜像还需 Docker
可选依赖	mosquitto (MQTT 数据面)；缺失时主链路不受影响

2.2 获取软件

2.2.1 方式一：源码构建

在仓库根目录执行：

```
make build
```

构建产物位于 `bin/` 目录：`bin/cloudcore`、`bin/edgecore`。`keadm` 可用 `go build -o bin/keadm ./cmd/keadm` 构建；`mock-cloudhub` 为联调工具，用 `go run ./hack/mock-cloudhub` 运行（无需构建）。

2.2.2 方式二：使用发布制品

发布目录 `release/v0.1.0/` 提供预编译二进制：`cloudcore`、`edgecore`、`keadm` 的 `linux-amd64` 与 `darwin-arm64` 版本，以及 Helm Chart 包 `edgeflow-0.1.0.tgz`。下载后可使用 `checksums.txt` 校验文件完整性，并将二进制放入 `PATH` 或仓库 `bin/` 目录后按需执行。

2.2.3 方式三：容器镜像

可自行构建镜像,或直接使用已发布的 `edgeflow/cloudcore:v0.1.0`、`edgeflow/edgecore:v0.1.0`。本地构建命令（仓库根目录执行）：

```
docker build -f build/docker/Dockerfile --target cloudcore -t edgeflow/cloudcore:v0.1.0 .
docker build -f build/docker/Dockerfile --target edgecore -t edgeflow/edgecore:v0.1.0 .
```

2.3 安装云端（cloudcore）

2.3.1 方式一：keadm 生成 Kubernetes 部署产物（推荐）

`keadm init` 在管理机上生成云端部署产物，产物可提交到 Kubernetes 集群执行。参数如下：

参数	默认值	说明
<code>--cloudcore-image</code>	<code>edgeflow/cloudcore:v0.1.0</code>	cloudcore 容器镜像
<code>--tls</code>	关	启用云边 mTLS，注入 TLS 相关环境变量
<code>--tls-san</code>	空	证书 SAN (Subject Alternative Name)，逗号分隔，如 <code>IP:1.2.3.4,DNS:host</code> ；仅 <code>--tls</code> 时生效
<code>--service-type</code>	<code>NodePort</code>	Service 类型：NodePort（边缘跨集群接入）或 ClusterIP（仅集群内访问）
<code>--output-dir</code>	<code>./keadm-out</code>	产物输出目录

生成命令示例：

```
# 最简（明文通道，本地/测试用）
keadm init --output-dir=./keadm-out

# 生产推荐：启用云边 mTLS，并注入证书 SAN（边缘节点用 IP 接入时必须覆盖访问地址）
keadm init --tls --tls-san=IP:192.168.1.10 --output-dir=./keadm-out
```

产物说明：

- `cloudcore.yaml`：包含 Deployment（副本 1、探针指向 `/healthz`、安全上下文 `nonroot`）与 Service（NodePort 类型，暴露 HTTP 8080 与 CloudHub 10000 两个端口；hub 端口由集群自动分配在 30000-32767）；
- `NOTES.txt`：部署步骤、验证方法、Helm 替代路径与 mTLS 说明。

在集群上执行产物：

```
kubectl apply -f cloudcore.yaml

# 验证
kubectl get deploy,svc,pods -l app.kubernetes.io/component=cloudcore
kubectl port-forward svc/edgeflow-cloudcore 8080:8080
curl http://127.0.0.1:8080/healthz          # 期望 HTTP 200

# 获取边缘节点接入用的 CloudHub 节点端口
kubectl get svc edgeflow-cloudcore -o jsonpath='{.spec.ports[?(@.name=="hub")].
  nodePort}'
```

2.3.2 方式二：Helm 部署

使用仓库内置 Chart 部署云端：

```
helm install edgeflow build/charts/edgeflow

# 集群外边缘节点接入时：开启 hub 端口并改用 NodePort
helm install edgeflow build/charts/edgeflow \
  --set service.hubEnabled=true --set service.type=NodePort
```

验证部署：

```
kubectl get deploy,svc,pods -l app.kubernetes.io/instance=edgeflow
kubectl port-forward svc/edgeflow-cloudcore 8080:8080
curl http://127.0.0.1:8080/healthz
```

关键配置项：`cloudcore.image.repository/tag`（镜像地址与版本，默认 `edgeflow/cloudcore:v0.1.0`）、`service.type`（默认 `ClusterIP`）、`service.hubEnabled`（默认 `false`，集群外边缘接入需开启并配合 `NodePort/LoadBalancer`）。

2.3.3 方式三：裸进程运行（单机联调）

开发或验证环境可直接运行二进制：

```
./bin/cloudcore
```

默认监听 REST API 8080 与 CloudHub 10000 端口，端口可通过环境变量或命令行覆盖（详见第 3 章 [chapter 三](#)）。此方式仅用于单机联调，生产环境请使用方式一或方式二。

2.4 接入边缘节点（edgecore）

`keadm join` 在管理机或边缘节点上生成边缘接入产物。参数如下：

参数	默认值	说明
<code>--cloudcore-ip</code>	必填	云端 CloudHub 节点 IP (IPv4/IPv6 均可)
<code>--cloudcore-port</code>	10000	CloudHub 端口; NodePort 部署时填集群分配的节点端口
<code>--token</code>	必填	接入令牌 (预留字段, 当前版本 edgecore 尚未消费)
<code>--node-id</code>	edge-< 主机名 >	边缘节点 ID, 仅允许字母、数字、连字符与下划线
<code>--tls</code>	关	启用 mTLS, 地址自动使用 wss://
<code>--output-dir</code>	./keadm-out	产物输出目录

生成命令示例:

```
keadm join --cloudcore-ip=192.168.1.10 --token=<token> --output-dir=./keadm-out

# TLS 集群 + NodePort 部署 (端口为集群分配的 hub 节点端口)
keadm join --cloudcore-ip=192.168.1.10 --cloudcore-port=31000 \
--token=<token> --node-id=edge-worker-01 --tls --output-dir=./keadm-out
```

产物说明:

- `edgecore.env`: 环境变量文件, 键名与 edgecore 读取的环境变量一一对应, 包括 `EDGEFLOW_EDGECORE_NO` `EDGEFLOW_EDGECORE_CLOUD_ADDR(ws://<ip>:<port>/v1/edge)`、`EDGEFLOW_EDGECORE_DB_PATH` (`/var/lib/edgeflow/edgeflow.db`)、`EDGEFLOW_EDGECORE_MQTT_ADDR(tcp://127.0.0.1:1883)` 等; `--tls` 时追加 TLS 与证书目录配置; `TOKEN` 为预留键;
- `edgecore.service`: systemd 单元文件, 通过 `EnvironmentFile=/etc/edgeflow/edgecore.env` 加载配置, `ExecStart=/usr/local/bin/edgecore`, 崩溃自动重启;
- `install.sh`: 一键安装脚本 (需 root), 安装二进制、环境文件与 systemd 单元并启用服务;
- `README.md`: 接入说明与手动安装片段。

在边缘节点上执行:

```
# 1. 将产物与 edgecore 二进制 (与节点架构匹配) 拷贝到边缘节点
# 2. 执行安装脚本 (需 root)
sudo ./install.sh

# 3. 验证
systemctl status edgecore
journalctl -u edgecore -f
```

mTLS 说明: 启用 `--tls` 后, edgecore 首次运行会在 `/etc/edgeflow/certs` 自动生成或加载证书 (幂等); 云端侧需保证服务端证书 SAN 覆盖边缘节点访问的地址, 否则云边连接会被拒绝。

2.5 部署验证

部署完成后，建议按以下清单验证：

- 云端 `/healthz` 返回 HTTP 200 与版本信息；
- 边缘节点已注册：云端查询 `/api/v1/nodes` 能看到节点且状态为 `Ready`（查询方法见第 4 章 [chapter 四](#)）；
- `systemctl status edgecore` 显示 `active (running)`，日志无致命错误；
- 可下发一个测试 Pod 验证端到端链路（见第 3 章 [chapter 三](#)）。

2.6 卸载与清理

```
# 清理 keadm 生成的产物（交互确认；--force 跳过确认，幂等）
keadm reset --output-dir=./keadm-out

# 卸载 Helm 部署
helm uninstall edgeflow

# 清理 Edged 管理的容器（按标签精确删除，不影响其他容器）
docker rm -f $(docker ps -aq --filter label=edgeflow.pod) 2>/dev/null

# 清理本地 SQLite 元数据（生产环境请按备份策略处理）
rm -f data/edgeflow.db
```

2.7 升级与回滚

keadm 在产物层面提供升级与回滚能力（执行前自动备份并记录操作台账）：

```
# 升级产物到新版本
keadm upgrade --version=v0.2.0 --operator=alice --output-dir=./keadm-out

# 回滚到最近一次备份
keadm rollback --latest --output-dir=./keadm-out

# 查询操作台账
keadm ops-ledger --limit=10
```

升级或回滚后需重新应用产物（`kubectl apply` 或 `./install.sh`）。注意：cloudcore 为内存态，云端重启后查询数据清空，边缘节点重连后会自动重新注册并恢复上报。

2.8 常见问题

现象	处理
缺少必填参数 <code>--cloudcore-ip</code> 或 <code>--token</code>	补全对应参数后重试
<code>--node-id</code> 含空白字符	使用 <code>--node-id=edge-xxx</code> 显式指定合法 ID
<code>kubectl apply</code> 报 schema 校验失败	检查 <code>kubectl</code> 版本与集群 API 版本是否兼容
edgecore 起不来	<code>journalctl -u edgecore -e</code> 查看日志；确认可达 <code>--cloudcore-ip</code> 的 hub 端口；确认 <code>env</code> 文件键值未被手改
云边连接被拒（TLS）	证书 SAN 未覆盖访问地址，云端以 <code>--tls-san=IP:<访问 IP></code> 重新 <code>init</code> 并 <code>apply</code>
Pod 容器未创建	确认 Docker daemon 运行（ <code>docker info</code> ）；镜像本地未缓存时首次拉取需要时间

第3章 快速入门

本章内容：在单台开发机上用约 10 分钟跑通 EdgeFlow 完整链路：构建制品、启动云端与边缘、注册节点、下发 Pod、查看容器、查看设备数据、下发设备指令。适合第一次接触 EdgeFlow 的操作人员。

3.1 准备工作

开始前请确认：

- 本机为 macOS 或 Linux；
- Docker 已安装且 daemon 运行中（Edged 的容器运行时）；
- 已安装 curl；源码构建还需 Go 1.26+ 与 Make；
- 已进入 EdgeFlow 仓库根目录（`cd edgeflow`）；
- 可选：mosquitto（MQTT 数据面），缺失时跳过不影响主链路。

```
docker info      # 确认 Docker daemon 运行
go version       # 确认 Go 版本（源码构建时需要）
```

3.2 第一步：构建制品

```
make build
```

构建产物位于 `bin/:bin/cloudcore.bin/edgecore`。也可以直接使用发布目录 `release/v0.1.0/` 中的预编译二进制（获取方式见第 2 章 [chapter 二](#)）。

3.3 第二步：启动云端 cloudcore

打开终端 A，执行：

```
./bin/cloudcore
```

预期日志输出 `HTTP server listening on :8080`。默认监听：REST API 端口 8080、CloudHub WebSocket 端口 10000。端口可通过环境变量覆盖（`EDGEFLOW_CLOUDCORE_PORT`、`EDGEFLOW_CLOUDCORE_HUB_PORT`），命令行参数 `--port` 优先级更高。

验证健康检查：

```
curl http://127.0.0.1:8080/healthz
```

预期返回 HTTP 200 与 `{"status":"ok","version":{...}}`。

3.4 第三步：启动边缘 edgecore

打开终端 B，执行：

```
EDGEFLOW_EDGECORE_NODE_ID=node-1 \  
EDGEFLOW_EDGECORE_CLOUD_ADDR=ws://127.0.0.1:10000/v1/edge \  
./bin/edgecore
```

说明：

- `EDGEFLOW_EDGECORE_NODE_ID`：节点 ID。不设置时默认为 `edge-< 主机名 >`，仅允许字母、数字、连字符与下划线；
- `EDGEFLOW_EDGECORE_CLOUD_ADDR`：云端 CloudHub 地址（明文通道用 `ws://`；启用 mTLS 时用 `wss://`，见第 2 章 [chapter 二](#)）；
- `edgecore` 启动后自动完成节点注册、云边连接，并随进程启动内置模拟温湿度传感器 `sensor-01` (`mock_sensor`)，无需额外配置。

3.5 第四步：验证节点注册

回到终端 A，查询节点列表：

```
curl http://127.0.0.1:8080/api/v1/nodes
```

预期返回 JSON 数组，包含 `nodeID` 为 `node-1`、`status` 为 `Ready` 的节点。若长时间未出现，请检查终端 B 的 `edgecore` 日志与网络连通性。

3.6 第五步：下发第一个 Pod (nginx)

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/podsync \  
-H 'Content-Type: application/json' \  
-d '{"operation":"add","pod":{"name":"nginx","namespace":"default","image":"nginx:1.25-alpine","replicas":1}}'
```

预期返回 `{"status":"ok","acked":true}`，表示边缘节点已确认。镜像首次拉取可能需要一些时间。

3.7 第六步：查看容器与 Pod 状态

在边缘侧查看容器（本机即边缘）：

```
docker ps --filter label=edgeflow.pod
```

预期看到容器 `edgeflow-default-nginx-0` 处于运行状态。

查询云端 Pod 状态：

```
curl http://127.0.0.1:8080/api/v1/pods
```

预期返回 phase 为 Running 的 Pod 记录。

3.8 第七步：查看设备数据

```
curl http://127.0.0.1:8080/api/v1/devices
```

预期返回设备 sensor-01, properties 中包含 temperature 与 humidity 两个属性 (周期上报的实时值)。设备默认上报周期为 30 秒, 可设置 EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL=3s 加速观察。

3.9 第八步：下发设备指令

向模拟传感器下发目标温度指令：

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/device-command \
-H 'Content-Type: application/json' \
-d '{"deviceName":"sensor-01","property":"targetTemp","value":25}'
```

预期返回 {"status":"ok","acked":true}。再次查询设备：

```
curl http://127.0.0.1:8080/api/v1/devices
```

预期 sensor-01 的 desired 中写入 {"targetTemp":25}, 表明指令已被边缘执行并写回云端设备状态。

3.10 一键 Demo (可选)

仓库提供端到端演示脚本, 自动完成上述全部步骤并输出 DEMO PASS:

```
bash examples/demo.sh
```

脚本自动完成: 构建、挑选空闲端口、启动 cloudcore 与 edgecore (运行时数据落在临时目录)、验证节点注册、下发 nginx Pod、验证容器与 Pod 状态、验证设备数据流、下发设备指令、可选 MQTT 数据面验证, 最后自动清理资源。脚本幂等, 可重复运行; 设置 EDGEFLOW_DEMO_SKIP_BUILD=1 可跳过构建步骤。

3.11 清理

```
# 1. 回收 Pod (边缘 Edged 会自动删除容器)
curl -X POST http://127.0.0.1:8080/api/v1/nodes/node-1/podsync \
-H 'Content-Type: application/json' \
-d '{"operation":"delete","pod":{"name":"nginx","namespace":"default"}}'
```

```
# 2. 停止 cloudcore 与 edgecore (终端 A/B 按 Ctrl+C, 优雅退出)

# 3. 兜底清理容器与本地元数据 (可选)
docker rm -f $(docker ps -aq --filter label=edgeflow.pod) 2>/dev/null
rm -f data/edgeflow.db
```

3.12 常见问题

现象	处理
节点未注册或状态非 Ready	查看 edgecore 日志; 确认 CloudHub 端口 (默认 10000) 可达、云边地址配置正确
podsync 返回 504 或长时间未确认	边缘未确认下发: 确认 edgecore 正在运行, 且下发路径中的节点 ID 与注册的 nodeID 完全一致
容器未创建	确认 Docker daemon 运行 (docker info); 镜像首次拉取需要时间
设备无数据上报	上报周期默认 30 秒, 可设置 EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL=3s 加速观察

第4章 云端操作指南

本章内容：介绍 cloudcore 对外提供的全部 REST API 端点——用途、请求示例与响应字段说明，以及监控指标（/metrics）与 API 认证的启用方法。操作人员可通过 curl 或任意 HTTP 客户端调用。

4.1 通用约定

- **Base URL**: `http://<cloudcore-ip>:8080` (端口可用 `--port` 或环境变量 `EDGEFLOW_CLOUDCORE_PORT` 覆盖)；
- **数据格式**: JSON；请求需携带 `Content-Type: application/json`，响应同样为 JSON；
- **时间戳**: Unix 毫秒（注册、心跳、上报时间等）；
- **List 风格**: `edgenodes`、`pods`、`devices` 端点采用 Kubernetes List 风格 (`kind/apiVersion/items`)，空数据编码为 `[]` 而非 `null`；`/api/v1/nodes` 为裸数组；
- **路径参数**: `{nodeID}` 为边缘节点 ID (edgecore 注册时上报，默认 `edge-<主机名>`)，必须与注册时的 `nodeID` 完全一致；
- **状态语义**: cloudcore 状态为内存态，重启即清空；边缘节点重连后自动重新注册并恢复上报，边缘侧元数据由 SQLite 持久化，不受影响。

端点总览：

方法	路径	说明	主要状态码
GET	/healthz	健康检查 (探针用)	200
GET	/api/v1/nodes	全部节点 (运行视角)	200
GET	/api/v1/nodes/{nodeID}	单节点详情	200 / 404
GET	/api/v1/edgenodes	全部节点 (CRD对象视角)	200
GET	/api/v1/edgenodes/{nodeID}	单节点 EdgeNode对象	200 / 404
GET	/api/v1/pods	全部节点Pod状态	200
GET	/api/v1/nodes/{nodeID}/pods	单节点Pod状态	200 / 404
GET	/api/v1/devices	全部设备状态	200
GET	/api/v1/nodes/{nodeID}/devices	单节点设备状态	200 / 404
POST	/api/v1/nodes/{nodeID}/pods/sync	下发Pod配置	200 / 400 / 404 / 502 / 504

4.2 健康检查

4.2.1 GET /healthz

用途：服务探活，返回云端运行状态与版本信息（Helm 部署的存活/就绪探针均指向此路径）。

```
curl http://127.0.0.1:8080/healthz
```

响应示例（HTTP 200）：

```
{"status": "ok", "version": {"version": "v0.1.0", "gitCommit": "c880bd9", "buildTime": "2026-08-14T19:08:17+0800", "goVersion": "go1.26.2"}}
```

4.3 节点 API

4.3.1 GET /api/v1/nodes ——全部节点（运行视角）

用途：获取全部已注册节点，直接返回节点元数据数组（按 nodeID 排序），用于快速了解节点在线状态与平台信息。

```
curl http://127.0.0.1:8080/api/v1/nodes
```

响应示例（HTTP 200，无节点时返回 []）：

```
[
  {
    "nodeID": "edge-node-1",
    "nodeName": "edge-node-1",
    "arch": "arm64",
    "os": "darwin",
    "edgecoreVersion": "version=v0.1.0 gitCommit=... goVersion=go1.26.2",
    "cpu": 8,
    "memory": 0,
    "ip": "127.0.0.1",
    "registeredAt": 1786705914423,
    "lastHeartbeatAt": 1786705914423,
    "status": "Ready"
  }
]
```

字段说明：

字段	类型	说明
nodeID	string	节点唯一 ID (edgecore 的 EDGEFLOW_EDGECORE_NODE_ID)
nodeName	string	云端分配的节点名 (当前与 nodeID 一致)
arch / os	string	edgecore 所在平台 (注册时上报)
edgecoreVersion	string	edgecore 版本串
cpu / memory	int / uint64	节点资源 (内存: Linux 上报实际值, 非 Linux 平台为 0)
ip	string	连接来源 IP
registeredAt / lastHeartbeatAt	int64	注册时间 / 最近心跳时间 (Unix 毫秒)
status	string	Ready / Unknown / Offline

4.3.2 GET /api/v1/nodes/{nodeID} ——单节点详情

用途: 查询指定节点。节点不存在时返回 404 与 {"error": "node not found", ...}。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1
```

4.3.3 GET /api/v1/edgenodes ——全部节点 (CRD 视角)

用途: 以 EdgeNode 对象 (apiVersion: edgeflow.io/v1alpha1) 形式获取节点, Kubernetes List 风格, 便于按 CRD 对象消费。

```
curl http://127.0.0.1:8080/api/v1/edgenodes
```

响应示例 (HTTP 200):

```
{
  "kind": "EdgeNodeList",
  "apiVersion": "edgeflow.io/v1alpha1",
  "items": [
    {
      "apiVersion": "edgeflow.io/v1alpha1",
      "kind": "EdgeNode",
      "metadata": {"name": "edge-node-1", "uid": "...", "creationTimestamp": "2026-08-14T19:11:54+08:00"},
      "spec": {"nodeID": "edge-node-1", "role": "edge"},
      "status": {"phase": "Running", "heartbeatTime": "...", "conditions": [{"type": "Ready", "status": "True"}]}
    }
  ]
}
```

关键字段: spec.nodeID (节点唯一标识)、spec.role (角色, 默认 edge)、status.phase

(Pending/Running/Offline/Unknown)、`status.conditions`（健康条件，如 Ready）。

4.3.4 GET /api/v1/edgenodes/{nodeID} ——单节点 EdgeNode 对象

用途：查询指定节点的 EdgeNode 对象；节点不存在时返回 404。

```
curl http://127.0.0.1:8080/api/v1/edgenodes/edge-node-1
```

4.4 Pod API

4.4.1 GET /api/v1/pods ——全部 Pod 状态

用途：获取云端保存的全部 Pod 状态（由边缘上报）。

```
curl http://127.0.0.1:8080/api/v1/pods
```

响应示例（HTTP 200）：

```
{
  "kind": "PodStatusList",
  "apiVersion": "v1",
  "items": [
    {
      "nodeID": "edge-node-1",
      "podName": "nginx",
      "namespace": "default",
      "phase": "Running",
      "message": "",
      "lastReconcileAt": 1786705732883
    }
  ]
}
```

字段说明：phase 取 Running / Stopped / Absent / Error / Unknown；message 为附加说明（如错误原因）；lastReconcileAt 为边缘最近一次调谐时间（Unix 毫秒）。

4.4.2 GET /api/v1/nodes/{nodeID}/pods ——单节点 Pod 状态

用途：查询指定节点的 Pod 状态。语义约定：节点不存在（从未注册）返回 404；节点存在但无 Pod 返回 200 与空 items。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1/pods
```


4.5 设备 API

4.5.1 GET /api/v1/devices ——全部设备状态

用途：获取全部设备的数字孪生状态：properties 为设备实际上报值，desired 为云端下发的期望值（设备上报不会覆盖 desired）。

```
curl http://127.0.0.1:8080/api/v1/devices
```

响应示例（HTTP 200，内置模拟传感器 sensor-01）：

```
{
  "kind": "DeviceStatusList",
  "apiVersion": "v1",
  "items": [
    {
      "nodeID": "edge-node-1",
      "deviceName": "sensor-01",
      "namespace": "default",
      "properties": {"humidity": 46.39, "temperature": 29.38},
      "desired": {"targetTemp": 25},
      "lastReportedAt": 1786705917423
    }
  ]
}
```

字段说明：properties / desired 均为 map[string]float64；lastReportedAt 为最近上报时间（Unix 毫秒）。

4.5.2 GET /api/v1/nodes/{nodeID}/devices ——单节点设备状态

用途：查询指定节点的设备状态。语义与单节点 Pod 查询一致：节点不存在返回 404；存在但无设备返回 200 与空 items。

```
curl http://127.0.0.1:8080/api/v1/nodes/edge-node-1/devices
```

4.6 下发类 API（可靠下发）

三个下发端点共用同一套可靠投递语义：消息进入 CloudHub 发送缓冲，由边缘 EdgeHub 接收并处理，处理完成后回确认（Ack）给云端，云端收到确认后返回 200；未确认时按 5 秒超时重试，最多尝试 3 次。

4.6.1 POST /api/v1/nodes/{nodeID}/podsync ——下发 Pod 配置

用途：向边缘节点下发 Pod 期望配置（新增/更新/删除），由边缘 Edged 调谐容器。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/podsync \
-H 'Content-Type: application/json' \
```

```
-d '{"operation":"add","pod":{"name":"nginx","namespace":"default","image":"nginx":1.25-alpine},"replicas":1}}'
```

请求体字段:

字段	类型	必填	说明
operation	string	是	add / update / delete (白名单校验)
pod.name	string	是	Pod 名称 (delete 时作为删除键之一)
pod.namespace	string	否	命名空间 (缺省 default)
pod.image	string	add/update 必填	容器镜像 (delete 不需要)
pod.replicas	int	否	副本数 (Edged 按此保证多副本)

响应示例 (HTTP 200, 边缘已确认):

```
{"status":"ok","acked":true}
```

操作说明: 边缘收到后, 容器按 **edgeflow-< 命名空间 >-< 名称 >-< 序号 >** 命名并打标签; **delete** 时按命名空间与名称删除元数据, Edged 回收对应容器。

4.6.2 POST /api/v1/nodes/{nodeID}/config-sync —— 下发配置

用途: 向边缘节点下发 ConfigMap/Secret 配置, 供边缘应用使用。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/config-sync \
-H 'Content-Type: application/json' \
-d '{"operation":"add","config":{"name":"app-config","namespace":"default","kind":"ConfigMap","data":{"key1":"value1"}}}'
```

请求体字段:

字段	类型	必填	说明
operation	string	是	add / update / delete
config.name	string	是	配置名称
config.namespace	string	否	命名空间 (缺省 default)
config.kind	string	是	ConfigMap / Secret (白名单校验)
config.data	map[string]string	add/update 必填	键值数据 (Secret 的 value 当前为明文存储, 生产需自行加密)

响应示例 (HTTP 200): {"status":"ok","acked":true}。

4.6.3 POST /api/v1/nodes/{nodeID}/device-command —— 下发设备指令

用途: 向边缘设备下发期望值指令, 由对应 Mapper 执行; 执行结果写入边缘设备孪生并随周期上报回云端。内置 **mock_sensor** 支持 **targetTemp** 与 **reset** 指令。

```
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/device-command \
-H 'Content-Type: application/json' \
```

```
-d '{"deviceName":"sensor-01","property":"targetTemp","value":25}'
```

请求体字段：

字段	类型	必填	说明
deviceName	string	是	目标设备名称（路由到对应 Mapper）
namespace	string	否	命名空间（缺省 default）
property	string	是	目标属性名（如 targetTemp）
value	float64	否	期望值（数值型）

响应示例（HTTP 200，边缘已确认且期望值已写入云端设备状态）：

```
{"status":"ok","acked":true}
```

下发成功后，通过 GET /api/v1/devices 可在 desired 字段中看到写入的期望值。

4.6.4 五态响应语义

三个下发端点统一使用以下响应语义：

状态码	响应体（示例）	含义
200	{"status":"ok","acked":true}	边缘已确认（Ack ok）
400	{"error":"operation and pod.name are required"}	请求非法：JSON 解析失败 / 缺必填字段 / operation 或 kind 不在白名单
404	{"error":"node offline or not registered"}	节点未注册或离线（消息未送达，无需重试）
502	{"error":"edge rejected ack"}	边缘明确拒绝：消息已送达但处理失败（重试无意义）
504	{"error":"ack timeout after retries"}	确认超时且重试耗尽：可能已送达但未确认（可重试，边缘侧有幂等去重）
500	{"error":"send failed"}	其他内部错误

4.7 监控指标

4.7.1 GET /metrics

用途：输出 Prometheus 文本格式指标，供 Prometheus 等监控系统抓取，观测云端运行状态。

```
curl http://127.0.0.1:8080/metrics
```

指标说明：

指标名	类型	含义
edgeflow_cloudcore_nodes_total	gauge	已注册边缘节点总数（含离线）
edgeflow_cloudcore_pods_total	gauge	云端 Pod 状态记录总数
edgeflow_cloudcore_devices_total	gauge	云端设备状态记录总数
edgeflow_cloudcore_http_requests_total	counter	云端 HTTP 请求累计数（按路由模式与状态码分桶）
edgeflow_cloudcore_active_connections	gauge	CloudHub 活跃连接数

4.8 API 认证

云端 API 认证默认关闭；生产环境建议开启。

4.8.1 启用认证

启动 cloudcore 前设置两个环境变量：

```
EDGEFLOW_CLOUDCORE_AUTH=on EDGEFLOW_CLOUDCORE_API_TOKEN=<你的令牌> ./bin/cloudcore
```

注意：EDGEFLOW_CLOUDCORE_AUTH=on 时必须同时设置 EDGEFLOW_CLOUDCORE_API_TOKEN，否则 cloudcore 拒绝启动。

4.8.2 调用方式

认证开启后，所有 /api/v1/* 请求必须携带请求头 **Authorization: Bearer <token>**：

```
curl -H 'Authorization: Bearer <你的令牌>' http://127.0.0.1:8080/api/v1/nodes

# 未携带或携带错误 token 时返回 401
curl http://127.0.0.1:8080/api/v1/nodes
```

说明：/healthz 与 /metrics 不参与认证，始终可访问，以保证探活与监控采集通道可用。

4.9 排障指南

现象	处理
返回 401	API 认证已开启但未携带或携带错误的 token；确认请求头 Authorization: Bearer <token> 正确
返回 400	请求体 JSON 非法或缺少必填字段、operation/kind 不在白名单；按响应中 error 提示修正
返回 404（下发类）	节点未注册或离线，消息未送达；先确认 edgecore 运行状态与节点 ID 拼写，无需重复重试
返回 502	消息已送达但被边缘拒绝；检查请求体是否符合边缘校验规则（如设备名、属性名是否正确）
返回 504	可能已送达但未确认（超时重试耗尽）；可稍后重试，边缘侧有幂等去重，不会重复执行
云端重启后查询为空	云端状态为内存态，重启即清空；边缘节点重连后会自动重新注册并恢复上报

第5章 边缘节点与自治运行

EdgeFlow 的边缘侧由一个常驻进程 **edgecore**（边缘核心）承载。它运行在边缘节点上，负责三件事：与云端建立并维持管理通道、按云端期望状态调谐本机容器、以及在本机维护设备影子与元数据。本章说明 edgecore 的接入、通信、自治行为，以及如何配置它。

5.1 接入与连接管理

5.1.1 注册

edgecore 启动后第一条消息是 **Register**（注册），云端 CloudHub 校验后回 **RegisterAck**（注册确认）。注册通过后，节点在云端注册表中可见，状态为 **Ready**。节点 ID 由环境变量 `EDGEFLOW_EDGECORE_NODE_ID` 指定；未设置时使用默认约定 `edge-< 主机名 >`。

```
# 指定节点 ID 与云端地址启动 edgecore
export EDGEFLOW_EDGECORE_NODE_ID=edge-worker-01
export EDGEFLOW_EDGECORE_CLOUD_ADDR=ws://192.168.1.10:10000/v1/edge
bin/edgecore
```

5.1.2 心跳保活

注册完成后，edgecore 以 **30 秒**为周期向云端发送心跳（Heartbeat）。云端 NodeController 每 **30 秒**扫描一次注册表，若某节点心跳停滞超过 **180 秒**（约 6 个心跳周期），将该节点标记为 **Offline**。因此：

- 心跳周期：固定 30s（云边契约常量，不可配置）；`EDGEFLOW_EDGECORE_REPORT_INTERVAL` 仅控制 Pod 状态上报周期；
- 云端超时阈值：`EDGEFLOW_CLOUDCORE_NODE_TIMEOUT`，默认 180s；
- 云端扫描周期：`EDGEFLOW_CLOUDCORE_NODE_SCAN_INTERVAL`，默认 30s。

5.1.3 断线重连

网络抖动或云端重启导致连接断开时，edgecore **自动重连**，无需人工干预：

- 指数退避：1s/2s/4s/...，上限 60s；
- 重连成功后重新执行注册流程，退避计时重置；
- 重连期间本地功能不受影响（见第 5 章第 5.4 节）。

5.2 可靠投递

云边之间的下行消息（Pod 配置、设备指令等）使用**可靠投递**语义，保证云端能确认消息确实被边缘处理：

1. 云端发送消息并登记到 **pending**（待确认）队列；
2. 边缘收到并处理成功后回 **Ack**；
3. 云端收到 Ack 后移出 pending 队列；
4. 超时（默认单次 5 秒）未确认，使用**同一消息 ID** 重试，最多 3 次；仍失败则向 API 调用方返回 504。

边缘侧对重复消息做**幂等去重**：同一消息 ID 处理过一次后不再重复执行，因此重试不会造成重复下发。API 层表现见第 九 章与附录 B。

5.3 Edged 应用调谐

Edged 是 edgecore 内置的容器运行时管理模块，对标 KubeEdge 的 Edged。它采用**声明式调谐**模型：云端下发期望状态（Pod 定义）后，Edged 周期性比对期望与实际情况，差异即收敛。

- 调谐周期：EDGEFLOW_EDGECORE_RECONCILE_INTERVAL，默认 5s；
- 启动时立即调谐一次，之后按周期循环；
- 调谐数据来自 MetaManager 落盘数据（SQLite），因此断网时调谐照常进行。

5.3.1 多副本 Pod

云端下发的 Pod 支持 **replicas** 多副本。Edged 每轮调谐将本机实际运行的副本数收敛到期望值：

- 实际副本 **少于**期望 → 逐个补齐拉起；
- 实际副本 **多于**期望 → 逐个清理收缩；
- 每轮调谐只处理一个副本的差异（批大小 1），逐轮滚动，避免瞬间抖动。

5.3.2 健康自愈与 CrashLoopBackOff

Edged 每轮调谐都会检查容器运行状态（Inspect）。容器非 Running（异常退出、被杀死等）时自动重启，无需人工介入：

- 每次重启累加 **RestartCount** 并上报云端；

- 连续异常重启达到 **3 次** 阈值后进入 **CrashLoopBackOff** 退避：退避时长 **30 秒**，退避期间不再反复拉起；
- 容器稳定运行 **60 秒** 后重置计数，恢复正常自愈节奏。

典型表现：退出型镜像（如 **busybox** 执行完即退出）会被反复拉起并进入退避，属预期行为；业务容器持续崩溃时请查看容器日志定位根因（见第 [九](#) 章）。

5.3.3 镜像漂移检测

Edged 在调谐时会比期望镜像与实际运行镜像：

- 已运行的容器镜像与期望不一致 → 判定为**镜像漂移**，自动重建：停止旧容器 → 用期望镜像重新拉起；
- 同一轮只重建 1 个漂移容器，逐轮滚动完成全部收敛；
- 该机制同时是滚动更新的基础：更新 Pod 定义中的镜像后，Edged 自动逐副本完成替换。

5.4 断网自治

自治是边缘计算的核心价值：云端不可达时，边缘业务不能停。

1. 云端断开后，已下发的容器**持续运行**，调谐、设备采集、本地指令照常工作；
2. 断网语义验证：自动化用例以 **60 秒短时断网** 模拟验证；**30 分钟级** 长时运行语义（M2 验收口径）需在真实环境长跑确认；
3. 重连成功后，edgcore 重新注册，并将离线期间的 Pod 状态、设备上报**自动同步**到云端，云端视图恢复一致。

注意：云端状态为**内存态**，云端进程重启后节点需重新注册（见第 [九](#) 章）。边缘侧元数据由 MetaManager 持久化到本机 SQLite，断网期间不丢失。

5.5 edgcore 环境变量配置

edgcore 全部通过环境变量配置，未设置的项使用默认值（表 [5.1](#)）。时长类变量（如调谐/上报周期）合法范围为 **1s 至 10m**，超出范围或非法值回退默认值并打印告警。

生产环境建议使用 `keadm join` 生成 `edgcore.env` 与 `edgcore.service`（systemd 单元），由安装脚本统一部署，避免手工拼写错误（见第 [七](#) 章升级回滚时的产物管理）。

表 5.1: edgecore 环境变量

变量	默认值	说明
EDGEFLOW_EDGECORE_NODE_ID	edge-< 主机名 >	节点 ID, 注册用
EDGEFLOW_EDGECORE_CLOUD_ADDR	ws://127.0.0.1:10000/v1/edge	云端 Cloud-Hub 地址
EDGEFLOW_EDGECORE_TLS	关	on 时启用 mTLS (地址自动 wss://)
EDGEFLOW_EDGECORE_CERT_DIR	data/certs/	证书目录
EDGEFLOW_EDGECORE_DB_PATH	内置默认	MetaManager SQLite 路径
EDGEFLOW_EDGECORE_MQTT_ADDR	tcp://127.0.0.1:1883	本机 MQTT broker 地址
EDGEFLOW_EDGECORE_RECONCILE_INTERVAL	5s	Edged 调谐周期
EDGEFLOW_EDGECORE_REPORT_INTERVAL	30s	Pod 状态上报周期
EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL	30s	设备状态上报周期
EDGEFLOW_EDGECORE_MQTT_CONNECT_TIMEOUT	5s	单次 MQTT 建连超时

第6章 设备接入

EdgeFlow 通过**设备影子** (DeviceTwin) 统一管理边缘设备：设备数据在边缘汇聚、周期上报云端；云端指令经边缘下发到设备。本章说明设备模型、指令链路、Mapper 框架（设备映射器）与数据面。

6.1 设备模型：影子

每台设备在边缘维护一份**影子** (Shadow / DeviceTwin)，包含两类属性：

properties (实际值) 设备实际上报的属性值（如温度、湿度），由 Mapper 采样写入，周期上报云端；

desired (期望值) 云端期望设备达到的属性值，通过指令下发写入，云端与边缘各存一份。

影子是边缘侧设备数据的**汇聚点**：无论设备通过何种协议接入（MQTT、Modbus...），云端看到的都是统一的影子视图。边缘默认每 `EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL`（30 秒）把影子快照上报云端。

6.2 设备指令下发链路

云端下发指令走**可靠投递**通道（见第 五 章），完整链路为：

1. 调用方 POST `/api/v1/nodes/{nodeID}/device-command`, 携带 `deviceName` 与 `property` (期望值)；
2. 云端经 CloudHub 将 DeviceCommand 消息可靠送达边缘 (Ack 确认后 API 返回 200)；
3. 边缘按 `deviceName` 路由到对应 Mapper 直接执行指令 (MQTT 模式下另订阅本地指令主题 `devices/<ns>/<device>/command`，支持数据面直接下发)，设备属性发生变化；
4. 边缘影子 `properties` 更新，下一上报周期同步到云端。

```
# 下发设备指令：把 sensor-01 的目标温度设为 25
curl -X POST http://127.0.0.1:8080/api/v1/nodes/edge-node-1/device-command \
-H 'Content-Type: application/json' \
-d '{"deviceName":"sensor-01","property":"targetTemp","value":25}'
# 期望响应: {"status":"ok","acked":true}
```

常见失败语义：节点离线 → 404（未送达，无需重试）；边缘拒绝 → 502（重试无意义）；确认超时 → 504（可重试，边缘幂等去重）。详见附录 B。

6.3 Mapper 框架

Mapper（设备映射器）负责把具体设备的协议翻译成 EdgeFlow 的统一设备模型。Mapper 注册到边缘的 **MapperRegistry**，按 **deviceName** 路由设备消息。v0.1.0 内置两个 Mapper（**modbus** 需显式启用，见下）：

6.3.1 mock_sensor：模拟传感器

内置模拟温湿度传感器，用于验证全链路，无需真实硬件：

- 设备名：sensor-01；
- 采集周期：每 2 秒波动一次温度/湿度；
- 数据面：MQTT（遥测主题 `devices/default/sensor-01/telemetry`）；
- 支持指令：如 `targetTemp`（目标温度）写入期望值。

6.3.2 Modbus TCP Mapper

通过 `goburrow/modbus` 客户端库（社区标准 Modbus 客户端）对接 Modbus TCP 设备：

- 设备名：mb-sensor-01（unit ID 1），采集温度/湿度保持寄存器；
- 自带 `modbussim` 模拟器，无需真实设备即可联调；
- 操作台账（op-ledger）：每次读写操作落 SQLite 台账，可查设备 ID、方向（up/down）与时间，便于审计排障；
- 指令链路与 `mock_sensor` 一致：云端下发 `DeviceCommand{deviceName:"mb-sensor-01",...}`，Mapper 按指令写寄存器。

启用方式：Modbus Mapper 为可选接入，必须显式设置环境变量 `EDGEFLOW_MODBUS_ADDR`（如 `127.0.0.1:15020` 指向本机模拟器，或 `<设备 IP>:502` 指向真实 Modbus 设备）后启动 `edgecore`，该 Mapper 才会注册；未设置时 `mb-sensor-01` 不存在。

6.3.3 Mapper 的两种工作模式

Mapper 由 `edgecore` 装配时统一注册，按 `EventBus` 是否可用分为两种模式：

MQTT 模式 `EventBus` 连接成功：遥测发布到 MQTT 主题、订阅设备指令主题，数据面完整（broker 掉线后 `paho` 自动重连、订阅自动恢复）；

纯本地模式 MQTT 不可用（未装配/连接失败降级）：采集照常写入影子、本地指令照常执行，仅 MQTT 数据面不可用；云边通道（`DeviceCommand/DeviceReport`）不受影响。

两种模式对云端完全透明：云端只看到影子视图，不感知边缘内部数据面形态。这也是第 九 章「`EventBus` 连接失败不必停机」的机制基础。

6.4 EventBus 数据面与降级

边缘内部通过 **EventBus** (MQTT, 本机 1883) 承载设备数据面：设备/Mapper 之间的遥测与指令走 MQTT 主题，云边管理消息仍走 WebSocket 通道，二者互不干扰。

表 6.1: EventBus 主题约定 (QoS 1)

主题	方向	用途
devices/<ns>/<device>/telemetry	设备 → 边缘	设备遥测上报
devices/<ns>/<device>/command	边缘 → 设备	指令下发
edgeflow/<module>/<event>	模块间	内部事件 (预留)

降级行为：MQTT broker 不可用或连接超时 (默认 5 秒) 时，Mapper 自动降级**纯本地模式**——设备采集与本地指令照常工作，仅数据面 MQTT 发布/订阅不可用；云边通道 (DeviceCommand/DeviceReport) 不受影响。已建立连接后的掉线会自动重连 (QoS 1, 订阅自动恢复)；若为**启动时**连接失败而降级，则降级是装配期决策，broker 恢复后需重启 edgecore 才能启用 MQTT 数据面。

```
# 订阅查看 sensor-01 遥测 (本机 mosquitto broker)
mosquitto_sub -p 1883 -t 'devices/default/sensor-01/telemetry' -v
# 通过 MQTT 数据面直接下发指令
mosquitto_pub -p 1883 -t 'devices/default/sensor-01/command' \
-m '{"deviceName":"sensor-01","property":"targetTemp","value":23}'
```

6.5 设备状态查询

云端提供统一查询入口，返回设备的 **properties** 与 **desired**：

```
# 全部设备状态
curl http://127.0.0.1:8080/api/v1/devices
# 单节点设备状态
curl http://127.0.0.1:8080/api/v1/nodes/{nodeID}/devices
```

- 节点不存在 → 404；
- 节点存在但无设备 → 200 + 空 items (可无分支遍历)；
- 开启 Token 认证时需携带 **Authorization: Bearer <token>** 请求头 (见第 八 章)。

6.6 设备接入检查清单

接入一台新设备时，按以下顺序核对：

1. **数据面：**设备/Mapper 与 broker (或本地模式) 连通，遥测主题有数据 (mosquitto_sub 可观测)；

2. **影子**: edgcore 日志出现 Mapper 注册成功, 影子 `properties` 有值;
3. **上报**: 等一个上报周期 (默认 30s), 云端 `GET /api/v1/devices` 可见设备;
4. **指令**: 下发一条 `device-command`, 确认 200 + 设备属性变化 + 期望值写入云端。

第7章 升级与回滚

EdgeFlow 的升级/回滚由安装管理 CLI `keadm` 提供，作用在 `keadm` 生成的产物文件层面 (`edgecore.env` / `edgecore.service` / `install.sh`)，配合操作台账 (`ops-ledger`) 实现可审计、可恢复的升级流程。部署在 Kubernetes 上的云端组件则走 Helm 路径。两条路径互不替代，本章分别说明。

7.1 机制总览

表 7.1: 升级回滚机制一览

能力	说明
备份模型	升级前把产物快照备份到 <code>backups/</code> (含 <code>manifest.json</code> 与每文件 <code>sha256</code>)
台账模型	每次操作 (含失败) 追加写 <code>ops-ledger.jsonl</code> ，逐行 JSON
版本标记	<code>edgecore.env</code> 内版本标记行， <code>join</code> 写入、 <code>upgrade</code> 更新、 <code>rollback</code> 恢复
数据隔离	不触碰数据目录 (SQLite 等) 与用户文件

边界: 本机制只保证离线产物可恢复; 镜像 tag 变更、edgecore 二进制替换、节点侧 `install.sh` 的实际执行不在产物级机制内，需结合发布流程执行。

7.2 升级前准备

1. 记录当前产物基线 (可选但推荐):

```
sha256sum $OUT/edgecore.env $OUT/edgecore.service $OUT/install.sh > baseline.sha
```

2. 如需在台账中记录操作人，设置环境变量:

```
export KEADM_OPERATOR=alice
```

未设置时台账记录 `unknown`。

7.3 升级: `keadm upgrade`

```
# 升级到 v0.2.0 (先备份 → 更新版本标记 → 写台账)
keadm upgrade --version=v0.2.0 --output-dir=./keadm-out

# 演练模式: 备份后模拟失败 (产物不会被修改)，用于演练回滚流程
keadm upgrade --version=v0.2.0 --simulate-failure --output-dir=./keadm-out
```

执行流程:

1. **校验版本格式**：必须为 `vX.Y.Z`（如 `v0.2.0`），非法或与当前版本相同 → 退出码 2，不产生备份；
2. **预检**：三个产物文件（`env/service/install.sh`）任一缺失 → 退出码 1；
3. **备份**：复制产物到 `backups/<id>/`（`<id>` 为到秒的时间戳，同秒追加 `-2`、`-3...`），写入 `manifest.json`（时间/版本/操作人/文件清单/sha256）；
4. **更新版本标记**：整行替换 `edgecore.env` 中的版本标记；
5. **写台账**：追加 `result=ok` 记录。

任一环节失败：退出码 1，台账记 `failed`，并提示「执行 `keadm rollback --latest` 可恢复」。备份目录只增不删，保留恢复路径与审计现场。

7.4 回滚：keadm rollback

```
# 取最新有效备份回滚
keadm rollback --latest --output-dir=./keadm-out

# 指定备份 id 回滚 (id 见 backups/ 目录名或台账)
keadm rollback --backup=20260814-190701 --output-dir=./keadm-out
```

执行流程：

1. **参数校验**：`--backup` 与 `--latest` 互斥，必须二选一，违规退出码 2；
2. **定位备份**：`--latest` 取时间戳最新（含序号后缀）；`--backup=<id>` 精确匹配，不存在 → 退出码 1；
3. **完整性校验**：`manifest.json` 可解析、字段完整、清单内文件存在且 `sha256` 一致；校验失败 → 报错并 **保留备份目录**；
4. **恢复**：逐文件复制回输出目录，回读校验内容一致，并强制恢复权限（`env 0600 / service 0644 / install.sh 0755`）；
5. **写台账**：追加 `result=ok`（`from=` 当前版本，`to=` 备份版本）。

失败兜底：任何自动恢复不可用的场景，备份目录保留完整快照，错误提示中给出逐文件人工恢复命令，例如：

```
cp backups/<id>/edgecore.env      <output-dir>/edgecore.env
cp backups/<id>/edgecore.service  <output-dir>/edgecore.service
cp backups/<id>/install.sh        <output-dir>/install.sh
```

7.5 台账查询: keadm ops-ledger

```
# 最近 20 条 (默认)
keadm ops-ledger --output-dir=./keadm-out
# 最近 N 条
keadm ops-ledger --limit=5 --output-dir=./keadm-out
```

输出为 JSON 数组, 逐行 JSON 原样保留, 机器可解析。记录字段: **ts** (RFC3339 时间)、**action** (upgrade/rollback)、**from/to** (版本)、**result** (ok/failed)、**operator** (操作人)、**note** (备份 id、失败原因等)。台账不存在时输出 [] 并以 0 退出; 损坏行跳过并附警告。

7.6 Helm 路径 (云端组件)

云端 cloudcore 若通过 Helm Chart (build/charts/edgeflow) 部署, 升级与回滚走标准 Helm 流程:

```
# 升级 (--install 兼容首次安装)
helm upgrade --install edgeflow build/charts/edgeflow \
  --set cloudcore.env.EDGEFLOW_CLOUDCORE_TLS=on \
  --set cloudcore.env.EDGEFLOW_CLOUDCORE_CERT_DIR=/data/certs

# 回滚到上一个版本
helm rollback edgeflow <rev>
```

版本标记约定: keadm join 生成的 edgecore.env 首行注释下含版本标记行, 与 keadm init 默认镜像 tag (edgeflow/cloudcore:v0.1.0) 对齐:

```
# keadm 产物版本: v0.1.0
```

旧版 join 产物无此标记时, upgrade 视当前版本为 unknown, 更新时自动追加标记行 (兼容升级路径)。rollback 恢复后标记行随文件内容一并还原。

7.7 端到端演练建议

```
keadm init --output-dir=$OUT
keadm join --cloudcore-ip=192.168.1.10 --token=abc123 \
  --node-id=edge-worker-01 --output-dir=$OUT
keadm upgrade --version=v0.2.0 --simulate-failure --output-dir=$OUT # 预期失败
  exit=1
keadm rollback --latest --output-dir=$OUT # 恢复
keadm upgrade --version=v0.2.0 --output-dir=$OUT # 真实升级
keadm rollback --latest --output-dir=$OUT # 真实回滚
sha256sum -c baseline.sha # 产物一致
keadm ops-ledger --output-dir=$OUT # 台账 4 条
```


注意：重复升级每次都会产生新备份（只增不删），长期使用请定期人工归档 **backups/** 目录。

第8章 安全与认证

EdgeFlow v0.1.0 的安全基线由三层构成：**云边通道 mTLS**（双向认证）、**管理 API Token 认证**（默认关闭）与**审计台账**（默认开启）。本章按层说明启用方式与运维要点。

8.1 云边通道 mTLS

云边 WebSocket 通道默认是明文 `ws://`（仅限可信网络/本机验证）。生产环境必须启用 **mTLS**（双向 TLS）：云端验证边缘证书、边缘验证云端证书与主机名，任何一侧校验失败连接在协议层之前被拒绝。

8.1.1 启用开关

表 8.1: mTLS 启用开关

组件	开关 (=on)	证书目录变量 (默认)
云端 CloudHub	EDGEFLOW_CLOUDCORE_TLS	EDGEFLOW_CLOUDCORE_CERT_DIR (data/certs/)
边缘 EdgeHub	EDGEFLOW_EDGECORE_TLS	EDGEFLOW_EDGECORE_CERT_DIR (data/certs/)

两侧必须同时开启：云端只开 TLS 会拒绝明文连接；边缘只开 TLS 无法通过明文云端的握手。边缘开启后连接地址自动归一化为 `wss://`。

```
# 云端
export EDGEFLOW_CLOUDCORE_TLS=on
export EDGEFLOW_CLOUDCORE_CERT_DIR=/data/certs
# 跨主机/集群部署必须注入 SAN (逗号分隔)，非法条目启动即报错 (fail-fast)
export EDGEFLOW_CLOUDCORE_TLS_SAN="IP:10.0.0.5,DNS:cloudcore.edgeflow.svc"
bin/cloudcore

# 边缘
export EDGEFLOW_EDGECORE_TLS=on
export EDGEFLOW_EDGECORE_CERT_DIR=/data/certs
export EDGEFLOW_EDGECORE_CLOUD_ADDR=wss://10.0.0.5:10000/v1/edge
bin/edgecore
```

关键约束：边缘连接的地址 (`wss://<host>:<port>`) 必须落在服务端证书 SAN 内，否则握手失败。未设置 SAN 时证书仅覆盖本机回环 (`127.0.0.1/localhost/cloudcore`)，mTLS 默认只在本机可用。

8.1.2 使用 keadm 生成 TLS 产物

```
# 云端产物：注入 TLS 开关 + 证书目录 + SAN
keadm init --tls --tls-san=IP:192.168.1.10 --output-dir=./keadm-out

# 边缘产物：注入 EDGEFLOW_EDGECORE_TLS=on + CERT_DIR，地址用 wss://
keadm join --cloudcore-ip=192.168.1.10 --cloudcore-port=31000 \
  --token=<token> --node-id=edge-worker-01 --tls --output-dir=./keadm-out
```

8.2 证书管理

8.2.1 证书布局

证书目录（云端与边缘各自 `data/certs/`）包含：

表 8.2: 证书文件布局

文件	用途	有效期	关键属性
ca.crt / ca.key	自签 CA	10 年	CN=edgeflow-ca, CertSign
cloudcore.crt / cloudcore.key	云服务端证书	1 年	SAN: 回环 + 注入值, EKU serverAuth
edgecore.crt / edgecore.key	边客户端证书	1 年	CN=edgeflow-<nodeID>, EKU clientAuth

私钥权限 0600，证书文件 0644。证书在组件首次以 TLS 启动时**自动生成**（幂等：已存在则加载校验，绝不重新生成）；也可用 `hack/gen-certs.sh` 预置。证书损坏/不完整 → **启动失败**（fail-fast，不降级）。

8.2.2 轮换（人工编排）

1. 签发新证书（先备份并删除旧文件，或使用新证书目录）；
2. 滚动重启组件，**先云后边**：云端加载新服务端证书，边缘重连时自动携带新客户端证书；
3. 验证新连接建立后，清除旧证书文件。

轮换 CA 需**全量重签**所有叶子证书（旧叶子证书随 CA 更换立即失效）。CA 私钥仅靠文件权限保护，生产建议离线签发后移出节点或接入 KMS。

8.3 管理 API Token 认证

管理 API（REST，8080 端口）默认**不认证**（向后兼容）。启用方式：

```
export EDGEFLOW_CLOUDCORE_AUTH=on
export EDGEFLOW_CLOUDCORE_API_TOKEN=<你的令牌> # 必填，未设置则启动失败
bin/cloudcore
```

- EDGEFLOW_CLOUDCORE_AUTH=on 且 API_TOKEN 未设置 → **启动失败** (fail-fast, 防止”开了认证却没令牌”的空转);
- 请求必须携带 **Authorization: Bearer <token>**;
- 缺失/非 Bearer 方案/令牌不匹配 → 401 + WWW-Authenticate: Bearer 响应头;
- 常量时间比较防时序侧信道; 认证通过者记为身份 **token** (单令牌模型, 持有令牌即管理员; 完整 RBAC 角色模型为后续版本)。

8.4 审计台账

管理 API 的所有操作默认写入审计台账:

- 路径: EDGEFLOW_CLOUDCORE_AUDIT_PATH, 默认 `data/audit-ledger.jsonl`;
- 格式: **JSONL** (逐行 JSON, 追加写, 不覆盖历史);
- 记录内容: 请求路径、方法、结果、身份(Token 认证下含 operator; 认证失败记录 **anonymous**);
- **审计失败不阻断 API**: 写盘失败只记错误日志, 业务请求照常处理 (可用性优先)。

```
# 查看审计台账 (每行一条记录)
tail -f data/audit-ledger.jsonl
```

8.5 安全基线小结

- 已实现: 双向认证 (云验边、边验云 + 主机名)、私钥 0600、强制 TLS 1.2+、证书损坏 fail-fast、Token 认证 (默认关)、审计台账 (默认开);
- 已知限制: 吊销 (CRL/OCSP) 未实现; CSR 审批流未实现; 证书 CN 与 nodeID 未绑定校验; 单令牌模型 (无多角色授权); 设备侧认证 token 为预留字段 (keadm join 写入但 edgecore 当前未消费)。

第 9 章 常见问题与故障排查

本章汇集 v0.1.0 实测与文档沉淀的常见问题。排障第一步永远是看日志，先定位故障在哪一层：云边通道、容器运行时、设备数据面，还是 API 层。

9.1 日志位置

表 9.1: 各组件日志位置

组件	日志位置
cloudcore	stdout / 日志文件（部署形态而定）
edgecore (systemd)	journalctl -u edgecore -e / -f 跟踪
edgecore (裸进程)	stdout
审计台账	data/audit-ledger.jsonl (云端)
操作台账	<output-dir>/ops-ledger.jsonl (keadm)

9.2 常见问题速查表

9.3 分层排查指引

9.3.1 云边通道层

1. 确认 edgecore 进程存活且已注册：

```
curl http://127.0.0.1:8080/api/v1/nodes
```

2. 节点存在但状态 **Offline**：心跳停滞超过 180s（约 6 个 30s 心跳周期）。检查网络、云端进程；
3. 节点不存在（404）：从未注册或注册失败。看 edgecore 日志中 Register/RegisterAck 是否成功；
4. 启用 mTLS 后连不上：先确认两侧开关都开、证书目录正确、边缘连接地址在服务端证书 SAN 内（见第 八 章）。

9.3.2 容器运行层

1. 调谐周期默认 5s，改动 Pod 定义后最多等一个周期生效；
2. 容器反复拉起并进入退避：区分「业务崩溃」与「退出型镜像」。CrashLoopBackOff 是保护机制，不是故障本身；

表 9.2: 常见问题与处置

现象	常见原因	处置
edgecore 起不来	网络不通 / 配置错误	<code>journalctl -u edgecore -e</code> 查看; 确认可达 <code>--cloudcore-ip</code> 的 hub 端口; 确认 env 文件键值未被手改
EventBus 连接失败	MQTT broker 未启动 / 地址错	查看日志 确认已降级本地模式 (边缘照常运行); 确认 broker 监听 1883; macOS 注意 broker 二进制在 <code>/opt/homebrew/sbin/mosquitto</code>
MQTT 建连超时	broker 无响应 / 网络隔离	调整 <code>EDGEFLOW_EDGECORE_MQTT_CONNECT_TIMEOUT</code> (默认 5s); 确认 <code>EDGEFLOW_EDGECORE_MQTT_ADDR</code>
容器不启动	Docker 未安装 / daemon 未启动	Edged 依赖 Docker: 确认 <code>docker info</code> 正常; daemon 启动后 Edged 下轮调谐 (默认 5s) 自动拉起
端口冲突	8080 / 10000 / 1883 被占用	<code>lsof -i :8080</code> 等定位占用进程; 改用 <code>EDGEFLOW_CLOUDCORE_PORT</code> / <code>HUB_PORT</code> / <code>EDGEFLOW_EDGECORE_MQTT_ADDR</code>
节点未注册 404	节点从未注册 / 已离线	检查 edgecore 是否在跑、注册是否成功; <code>GET /api/v1/nodes</code> 确认节点状态
下发 504 超时	边缘宕机 / 链路抖动	确认边缘进程存活; 504 可重试, 边缘侧幂等去重; 持续 504 检查网络与心跳
容器反复重启	业务崩溃 / 退出型镜像	观察 <code>RestartCount</code> 与 <code>CrashLoopBackOff</code> (3 次阈值/30s 退避/ 60s 稳定重置); 查看容器日志定位业务根因
云端重启后节点消失	云端状态为内存态	边缘自动重连并重新注册; 如长时间未恢复, 重启 edgecore 触发重连

3. 镜像漂移检测会在期望镜像与运行镜像不一致时自动重建（每轮 1 个，逐轮滚动），滚动期间短暂重启属预期。

9.3.3 设备数据面层

1. GET /api/v1/devices 看不到设备：确认 edgecore 内 Mapper 已注册（日志有「Mapper 注册完成」），MQTT 模式下确认 broker 可达；
2. 设备属性不更新：确认采集周期（mock_sensor 默认 2s）与上报周期（默认 30s）；
3. 指令下发失败：按状态码区分——404 未送达（节点离线，修通道）、502 送达被拒（改请求）、504 未确认（重试）。

9.4 API 错误语义速记

- 404：节点未注册/离线，**无需重试**，先修通道；
- 502：消息已送达但边缘拒绝，**重试无意义**，检查请求内容；
- 504：可能送达但未确认，**可重试**，边缘幂等去重；
- 错误响应统一为 JSON：{"error": "< 机器可读原因 >"}。

完整语义见附录 B。

9.5 环境相关的已知边界

- **macOS/Docker Desktop**：宿主机无法直接路由 kind 节点 IP 的 NodePort（实测 connection reset），用 `kubect1 port-forward` 替代；Linux 上可直接 `nodeIP:nodePort` 访问；
- **真实多节点集群**：v0.1.0 验证范围为 kind 单节点集群 + 单边缘节点（2026-08-14 实测），多节点（10+）E2E 与压测未做；
- **镜像仓库**：镜像未推送远端仓库，跨机部署需自建 registry（`docker run -d -p 127.0.0.1:5001:5000 registry:2 + buildx` 双架构构建，见 docs/MULTIARCH.md）。

第 A 章 附录 A 环境变量参考

以下为 EdgeFlow v0.1.0 的主要环境变量(含组件装配与运维所需全部变量;EDGEFLOW_MODBUS_ADDR 见第 6 章)。设置方式: 启动进程前 `export`, 或由 `keadm join` 生成的 `edgecore.env` 统一注入。

A.1 cloudcore 组

表 A.1: cloudcore 环境变量

变量	默认值	说明
EDGEFLOW_CLOUDCORE_PORT	8080	管理 API 监听端口
EDGEFLOW_CLOUDCORE_HUB_PORT	10000	CloudHub (云边通道) 端口
EDGEFLOW_CLOUDCORE_TLS	关	on 启用 mTLS
EDGEFLOW_CLOUDCORE_CERT_DIR	data/certs/	证书目录
EDGEFLOW_CLOUDCORE_TLS_SAN	空 (仅回环)	证书 SAN, 逗号分隔, 非法 fail-fast
EDGEFLOW_CLOUDCORE_NODE_SCAN_INTERVAL	30s	节点健康扫描周期
EDGEFLOW_CLOUDCORE_NODE_TIMEOUT	180s	心跳超时阈值 (约 6 个心跳)
EDGEFLOW_CLOUDCORE_AUTH	关	on 启用 Token 认证
EDGEFLOW_CLOUDCORE_API_TOKEN	空	API 令牌; 认证开启时必须填, 否则启动失败
EDGEFLOW_CLOUDCORE_AUDIT_PATH	data/audit-ledger.jsonl	审计台账路径 (JSONL 追加写)

A.2 edgecore 组

时长类变量 (调谐/上报周期等) 合法范围 **1s 至 10m**, 超出或非法回退默认值并告警。

表 A.2: edgecore 环境变量

变量	默认值	说明
EDGEFLOW_EDGECORE_NODE_ID	edge-< 主机名 >	节点 ID
EDGEFLOW_EDGECORE_CLOUD_ADDR	ws://127.0.0.1:10000/v1/edge	云端 Cloud-Hub 地址
EDGEFLOW_EDGECORE_TLS	关	on 启用 mTLS (wss://)
EDGEFLOW_EDGECORE_CERT_DIR	data/certs/	证书目录
EDGEFLOW_EDGECORE_DB_PATH	内置默认	MetaManager SQLite 路径
EDGEFLOW_EDGECORE_MQTT_ADDR	tcp://127.0.0.1:1883	本机 MQTT broker 地址
EDGEFLOW_EDGECORE_RECONCILE_INTERVAL	5s	Edged 调谐周期
EDGEFLOW_EDGECORE_REPORT_INTERVAL	30s	Pod 状态上报周期
EDGEFLOW_EDGECORE_DEVICE_REPORT_INTERVAL	30s	设备状态上报周期
EDGEFLOW_EDGECORE_MQTT_CONNECT_TIMEOUT	5s	单次 MQTT 建连超时
EDGEFLOW_EDGECORE_TOKEN	空	预留: keadm join 写入,

A.3 keadm / 开发脚本组

表 A.3: keadm 与开发脚本环境变量

变量	默认值	说明
KEADM_OPERATOR	unknown	keadm 操作台账记录的操作人
EDGEFLOW_CLUSTER_NAME	edgeflow-dev	开发脚本 kind 集群名 (<code>hack/dev-up.sh</code>)
EDGEFLOW_EDGE_NODES	1	开发脚本边缘模拟容器数量
EDGEFLOW_EDGE_IMAGE	alpine:3.20	开发脚本边缘容器镜像
EDGEFLOW_EDGE_ARGS	空	开发脚本追加给 edgecore 的参数

后四个变量仅用于开发环境脚本 `hack/dev-up.sh` / `dev-down.sh`，不影响生产部署。

第 B 章 附录 B API 状态码语义

管理 API (REST, 默认 8080) 状态码语义:

记忆要点: 404 = 没送达; 502 = 送达但被拒; 504 = 可能送达但未确认。错误响应统一为 JSON: `{"error": "< 机器可读原因 >", ...}`。

表 B.1: API 状态码语义

状态码	含义
200	成功；下发类接口表示 边缘已确认 （Ack ok），响应 {"status":"ok","acked":true}
400	请求非法：JSON 解析失败 / 缺必填字段 / operation 或 kind 不在白名单
401	未授权：Token 认证开启且请求未携带/携带错误令牌（附 WWW-Authenticate: Bearer）
404	节点未注册或离线（ErrNodeOffline）；单资源查询不存在
500	内部错误：消息构建失败、发送通道异常等兜底
502	边缘明确拒绝（回 error Ack）：消息已送达但处理失败

处
置
建
议

—
修
正
请
求
体
携
带
Authori
Bearer
<token>

无
需
重
试，
先
修
云
边
通
道
查
看
cloud-
core
日
志，
反
馈
问
题
重
试
无
意
义，
检
查

第 C 章 附录 C 术语表

CloudCore (云端核心) 云端主进程, 含 CloudHub (云边通道服务端)、NodeController (节点健康扫描)、设备状态存储与 REST API。

EdgeCore (边缘核心) 边缘节点常驻进程, 含 EdgeHub (云边通道客户端)、Edged、Meta-Manager、DeviceTwin、Mapper 框架与 EventBus。

Edged 边缘容器运行时管理模块: 声明式调谐 (默认 5s 周期)、多副本、健康自愈与镜像漂移检测。

MetaManager 边缘元数据管理: KV + 节点 + Pod 数据持久化到本机 SQLite (WAL), 断网自治的数据基础。

DeviceTwin (设备影子) 每台设备的数字孪生: `properties` (实际值) + `desired` (期望值)。

Mapper (设备映射器) 把具体设备协议翻译为统一设备模型的组件; v0.1.0 内置 `mock_sensor` 与 Modbus TCP 两个 Mapper。

PodSync 云端向边缘可靠下发 Pod 配置的链路 (add/update/delete)。

ConfigSync 云端向边缘可靠下发 ConfigMap/Secret 配置的链路。

NodeJob 云端任务管理 (任务分发 CRD): 已决策关闭 (v0.1.0 范围外, 协议占位标注)。

mTLS 双向 TLS: 云端验证边缘证书、边缘验证云端证书与主机名, 云边通道默认实现。

影子设备 云端视角的设备逻辑实体, 由边缘影子周期上报汇聚而成。

EventBus 边缘内部 MQTT 设备通信总线 (数据面), broker 不可用时 Mapper 降级纯本地模式。

op-ledger Modbus Mapper 的读写操作台账 (SQLite), 记录设备 ID、方向与时间。

ops-ledger keadm 升级/回滚操作台账 (`ops-ledger.jsonl`)。

第 D 章 附录 D 版本与变更记录

D.1 v0.1.0 (2026-08-14, MVP 首发)

- 发布内容: 云边通信 (WebSocket 注册/心跳/指数退避重连/可靠投递)、边缘自治 (Edged 声明式调谐/多副本/健康自愈/镜像漂移检测/断网自治)、设备管理 (DeviceTwin/Map-per/EventBus/Modbus)、生产加固 (mTLS/多架构镜像/Helm Chart/keadm 安装 CLI/升级回滚)、发布文档 (API 规范/部署指南/示例);
- 质量门: 24 个包 `go test -race` 全绿; 总覆盖率 **77.8%** (门槛 70%); `golangci-lint` 0 issues; P0/P1 审查发现 0; 端到端示例 `examples/demo.sh` 通过 3 次;
- 制品: `release/v0.1.0/` (6 二进制 + Chart 包 + checksums.txt + sbom.json + im-ages.json) ; 镜像 `edgeflow/cloudcore:v0.1.0` / `edgeflow/edgecore:v0.1.0` (lin-ux/amd64 + arm64)。

D.2 已实现 vs 即将上线 / 范围外

表 D.1: 能力对照	
类别	内容
已实现	云边通信、边缘自治、设备管理、mTLS、Token 认证（默认关）、审计台账、keadm (init/join/upgrade/rollback/ops-ledger/reset/version)、Helm Chart、多架构镜像、/metrics 可观测性
即将上线 / 范围外	NodeJob 任务分发（已关闭）、完整 RBAC 角色模型（仅 Token 单令牌）、设备认证 token 消费（预留未做）、Flannel、边缘调度/资源超卖、OPC-UA 适配器、API 兼容矩阵、镜像安全扫描、消息压缩/ Protobuf、灰度发布

第 E 章 附录 E 已知限制与边界

1. **云端状态为内存态**: 云端进程重启后节点状态清空, 边缘需重新注册 (自动重连触发); 在途未确认消息可能丢失, 由上层控制器恢复后重新下发。
2. **镜像未推送远端仓库**: 镜像仅本地/自建 registry 可用, GitHub 远端 CI(lint+test+release) 未在真实远端运行。
3. **GitHub remote 未配置**: 配置 SSH key 后 `git remote add origin` 推送即可启用远端 CI。
4. **真实多节点集群未验证**: v0.1.0 在 kind 单节点集群 + 单边缘节点跑通 (2026-08-14 实测); 多节点 (10+) E2E 与 100 节点压测需真实集群环境, 未执行。
5. **macOS 本机验证限制**: 宿主机无法直接路由 kind 节点 IP 的 NodePort (实测 connection reset), 需 `kubect1 port-forward`; mTLS 默认证书仅覆盖回环地址, 跨主机必须注入 SAN。
6. **keadm 产物级升级边界**: 备份/恢复只覆盖 `env/service/ install.sh` 三个产物文件, 不触碰数据目录与用户文件; 真实部署 (镜像 tag、edgecore 二进制) 升级需结合发布流程。
7. **证书管理**: 吊销 (CRL/OCSP)、CSR 审批流、证书 节点 CN 强绑定未实现; CA 私钥仅文件权限保护。
8. **手册更新方式**: 本手册 LaTeX 工程纳入 Git 仓库; 修改 `docs/manual/` 下章节文件后, 用 `xelatex` 编译主文件生成 PDF (ctexbook 文档类)。